

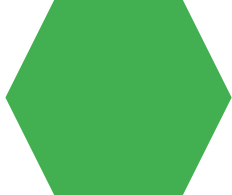


**PROJECT TITLE – SEASONAL AGRICULTURE
PERFORMANCE ANALYSIS**

STUDENT NAME – VIJI P

COLLEGE NAME – SOLAMALAI COLLEGE OF ENGINEERING

AICTE ID –STU6A68475D9F23E1785218909



PROBLEM STATEMENT

- Agricultural activities are influenced by seasonal variations in environmental conditions, farming practices, resource availability and market conditions. As a result, agricultural performance may differ from one season to another.
- However, raw agricultural data does not clearly explain how agricultural performance changes across seasons or what patterns can be observed in different seasonal conditions.
- The problem is to analyze the given agricultural dataset and investigate seasonal differences in agricultural performance by identifying meaningful patterns, trends, relationships and variations within the available data.



Project Description

- This project analyzes the Seasonal Agriculture Performance Dataset to study how yield, resource usage, and profit vary across Kharif, Rabi, and Zaid seasons.
- Using Python (Pandas, Seaborn, SciPy) in Google Colab, the data was cleaned, visualized, and statistically tested (ANOVA) to find seasonal patterns and give data-driven recommendations for better agricultural planning.

WHO ARE THE END USERS?

- Farmers
- Agricultural policy makers
- Agri-business/supply chain companies
- Agri-finance (banks/insurance)
- Researchers & students



Technology Used

- Programming Language: Python
- Platform: Google Colab (Jupyter Notebook environment)
- Data Handling: Pandas, NumPy
- Data Visualization: Matplotlib, Seaborn
- Statistical Analysis: SciPy (one-way ANOVA test)
- Data Format: CSV (Comma-Separated Values)
- Documentation: Jupyter Notebook (.ipynb)



RESULTS

Code

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('seasonal_agriculture_performance_dataset.csv')

df.head()
```

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving seasonal_agriculture_performance_dataset.csv to seasonal_agriculture_performance_dataset.csv

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	...	Seed_Quality_Score	Yield_Tonnes_Ha	Production_Inr_Tonne
0	SF10001	Andhra Pradesh	Rajkot	Wheat	Kharif	0.53	486.4	24.3	69.1	5.8	...	0.76	2.46	1.23
1	SF10002	Maharashtra	Nalgonda	Maize	Kharif	6.53	855.4	25.7	78.4	5.1	...	0.94	0.30	1.23
2	SF10003	Telangana	Warangal	Pulses	Rabi	4.86	455.6	23.9	56.4	10.3	...	0.98	0.52	1.23
3	SF10004	Telangana	Indore	Rice	Kharif	4.50	753.2	29.7	65.2	6.3	...	0.74	1.84	1.23
4	SF10005	Karnataka	Ludhiana	Maize	Zaid	4.21	101.6	34.1	55.2	6.5	...	0.90	2.21	1.23

5 rows x 28 columns

Code

```
df.tail()

df.shape

df.columns

Index(['Farm_ID', 'State', 'District', 'Crop', 'Season', 'Farm_Area_Hectares', 'Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct', 'Sunlight_Hours_Day', 'Soil_pH', 'Soil_Moisture_pct', 'Nitrogen_kg_ha', 'Phosphorus_kg_ha', 'Potassium_kg_ha', 'Irrigation_Method', 'Fertilizer_kg_ha', 'Pesticide_Litre_ha', 'Seed_Quality_Score', 'Yield_Tonnes_Ha', 'Production_Tonnes', 'Market_Price_INR_Tonne', 'Total_Cost_INR', 'Revenue_INR', 'Profit_INR', 'Water_Used_m3', 'Water_Efficiency_t_per_1000m3', 'Disease_Pest_Risk_pct'], dtype='object')
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	...	Seed_Quality_Score	Yield_Tonnes_Ha	Production_Inr_Tonne
3995	SF13996	Karnataka	Rajkot	Wheat	Kharif	4.72	865.4	27.3	51.4	6.3	...	0.74	0.51	1.23
3996	SF13997	Karnataka	Krishna	Chilli	Kharif	8.77	947.5	29.2	85.5	6.1	...	0.80	2.75	1.23
3997	SF13998	Andhra Pradesh	Erode	Pulses	Zaid	10.47	80.0	35.4	68.5	8.6	...	0.99	0.62	1.23
3998	SF13999	Punjab	Warangal	Cotton	Kharif	1.73	475.0	28.2	61.7	6.4	...	0.98	1.35	1.23
3999	SF14000	Punjab	Nashik	Wheat	Kharif	3.31	984.3	25.8	77.2	5.9	...	0.99	2.36	1.23

5 rows x 28 columns

Code

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4000 entries, 0 to 3999
Data columns (total 28 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Farm_ID                               4000 non-null   object
1   State                                 4000 non-null   object
2   District                             4000 non-null   object
3   Crop                                  4000 non-null   object
4   Season                                4000 non-null   object
5   Farm_Area_Hectares                    4000 non-null   float64
6   Rainfall_mm                           3952 non-null   float64
7   Avg_Temperature_C                     4000 non-null   float64
8   Humidity_pct                          4000 non-null   float64
9   Sunlight_Hours_Day                    4000 non-null   float64
10  Soil_pH                               4000 non-null   float64
11  Soil_Moisture_pct                      3960 non-null   float64
12  Nitrogen_kg_ha                         4000 non-null   float64
13  Phosphorus_kg_ha                      4000 non-null   float64
14  Potassium_kg_ha                       4000 non-null   float64
15  Irrigation_Method                      4000 non-null   object
16  Fertilizer_kg_ha                       4000 non-null   float64
17  Pesticide_Litre_ha                    4000 non-null   float64
18  Seed_Quality_Score                     4000 non-null   float64
19  Yield_Tonnes_Ha                       3968 non-null   float64
20  Production_Tonnes                     4000 non-null   float64
21  Market_Price_INR_Tonne                4000 non-null   int64
22  Total_Cost_INR                        4000 non-null   int64
23  Revenue_INR                           4000 non-null   int64
24  Profit_INR                            4000 non-null   int64
25  Water_Used_m3                         4000 non-null   int64
26  Water_Efficiency_t_per_1000m3         4000 non-null   float64
27  Disease_Pest_Risk_pct                  4000 non-null   float64
```

Index	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	Soil_pH	Soil_Moisture_pct	Nitrogen_kg_ha	Phosphorus_kg_ha	Potassium_kg_ha	Irrigation_Method	Fertilizer_kg_ha	Pesticide_Litre_ha	Seed_Quality_Score	Yield_Tonnes_Ha	Production_Tonnes	Market_Price_INR_Tonne	Total_Cost_INR	Revenue_INR	Profit_INR	Water_Used_m3	Water_Efficiency_t_per_1000m3	Disease_Pest_Risk_pct	
0	SF10001	Andhra Pradesh	Rajkot	Wheat	Kharif	0.53	3952.000000	4000.000000	4000.000000	4000.000000	4000.000000	3960.000000	4000.000000	4000.000000	4000.000000	4000.000000	4000.000000	4000.000000	4000.000000	0.76	2.46	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
1	SF10002	Maharashtra	Nalgonda	Maize	Kharif	6.53	601.152657	26.81890	63.210500	7.323050	6.701010	26.508232	120.250825	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.94	0.30	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
2	SF10003	Telangana	Warangal	Pulses	Rabi	4.86	288.925025	3.81818	11.960834	1.121186	0.641461	6.712484	31.216452	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.98	0.52	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
3	SF10004	Telangana	Indore	Rice	Kharif	4.50	80.000000	16.20000	25.000000	3.500000	5.200000	8.000000	40.000000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.74	1.84	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
4	SF10005	Karnataka	Ludhiana	Maize	Zaid	4.21	370.200000	23.90000	54.775000	6.600000	6.260000	21.700000	98.500000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.90	2.21	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
5	SF10006	Karnataka	Ludhiana	Maize	Zaid	4.21	582.000000	26.90000	63.000000	7.300000	6.690000	26.400000	120.100000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.90	2.21	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
6	SF10007	Karnataka	Ludhiana	Maize	Zaid	4.21	834.950000	29.60000	71.900000	8.100000	7.140000	31.300000	141.225000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.90	2.21	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23
7	SF10008	Karnataka	Ludhiana	Maize	Zaid	4.21	1395.200000	39.70000	95.000000	11.000000	8.400000	47.000000	220.000000	400.000000	400.000000	400.000000	400.000000	400.000000	400.000000	0.90	2.21	1.23	1.23	1.23	1.23	1.23	1.23	1.23	1.23

RESULTS

```
[ ] df.isnull().sum()
```

...	0
Farm_ID	0
State	0
District	0
Crop	0
Season	0
Farm_Area_Hectares	0
Rainfall_mm	48
Avg_Temperature_C	0
Humidity_pct	0
Sunlight_Hours_Day	0
Soil_pH	0
Soil_Moisture_pct	40
Nitrogen_kg_ha	0
Phosphorus_kg_ha	0
Potassium_kg_ha	0
Irrigation_Method	0
Fertilizer_kg_ha	0
Pesticide_Litre_ha	0

```
df.duplicated().sum()

np.int64(0)

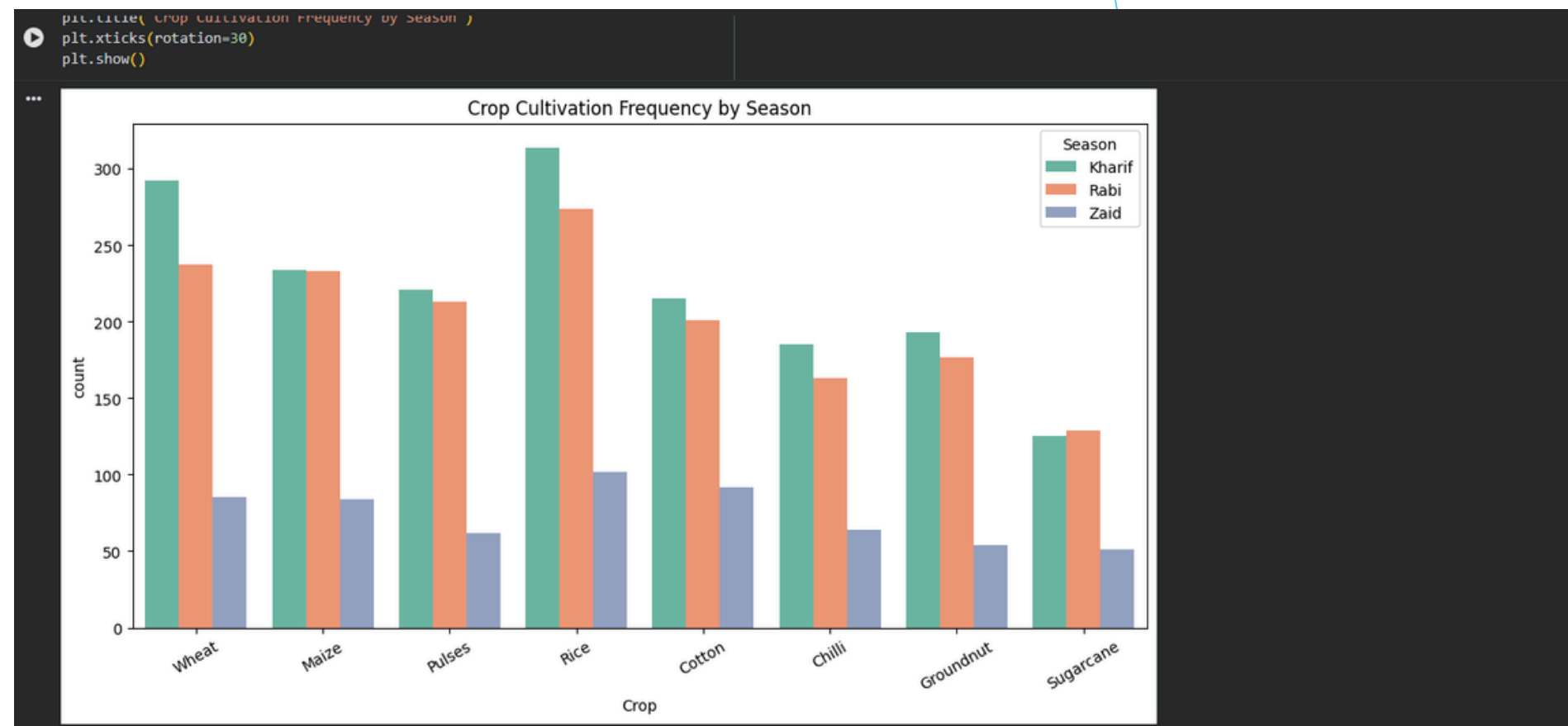
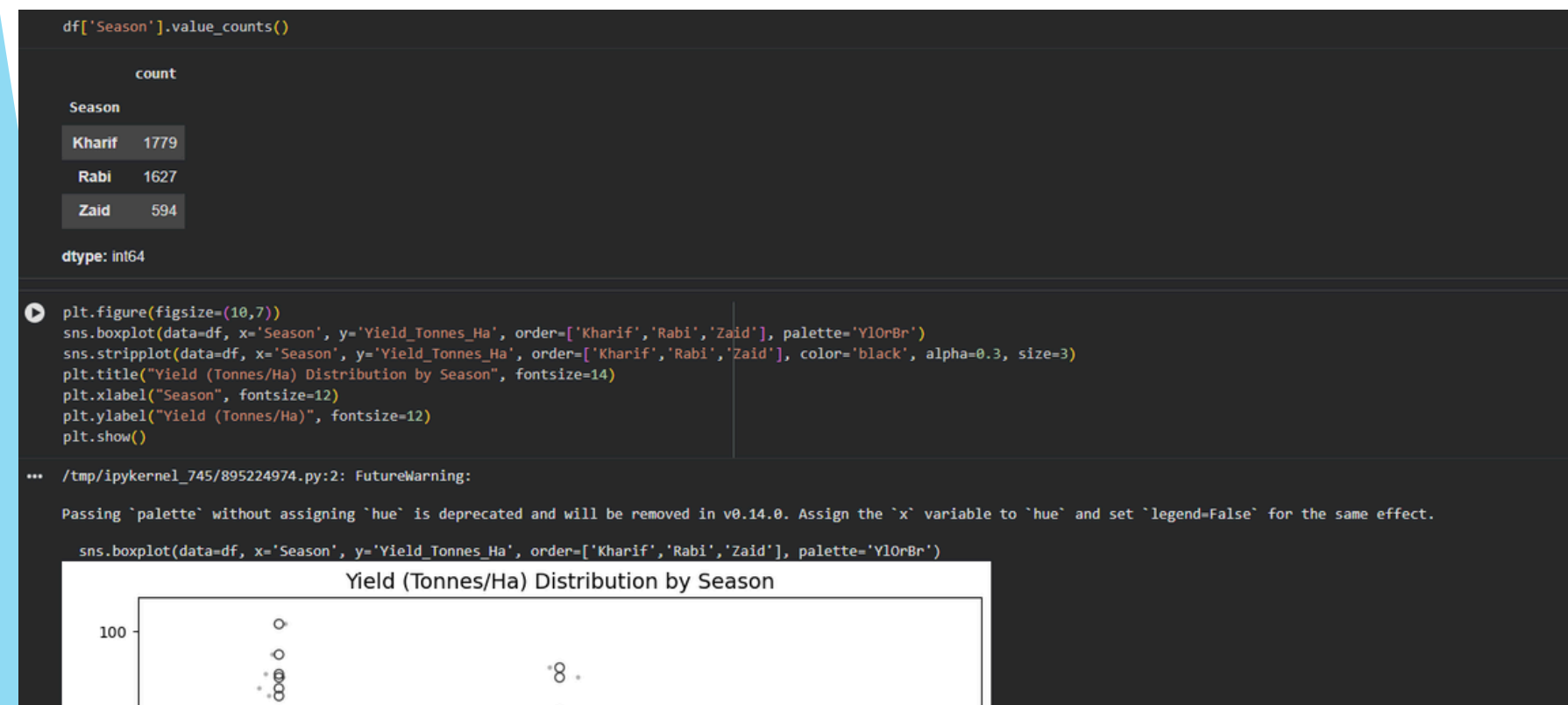
df.sample(5)
```

	Farm_ID	State	District	Crop	Season	Farm_Area_Hectares	Rainfall_mm	Avg_Temperature_C	Humidity_pct	Sunlight_Hours_Day	...	Seed_Quality
3863	SF13864	Madhya Pradesh	Nashik	Rice	Kharif	7.96	868.9	25.6	63.2	5.5	...	...
2408	SF12409	Punjab	Nashik	Pulses	Rabi	5.10	232.4	28.0	53.0	8.7	...	...
3297	SF13298	Madhya Pradesh	Nashik	Rice	Kharif	4.30	954.2	26.1	86.5	5.5	...	...
3871	SF13872	Andhra Pradesh	Indore	Chilli	Kharif	6.43	784.8	31.9	68.1	6.4	...	...
1089	SF11090	Maharashtra	Ludhiana	Sugarcane	Rabi	7.05	195.2	26.6	62.7	8.4	...	...

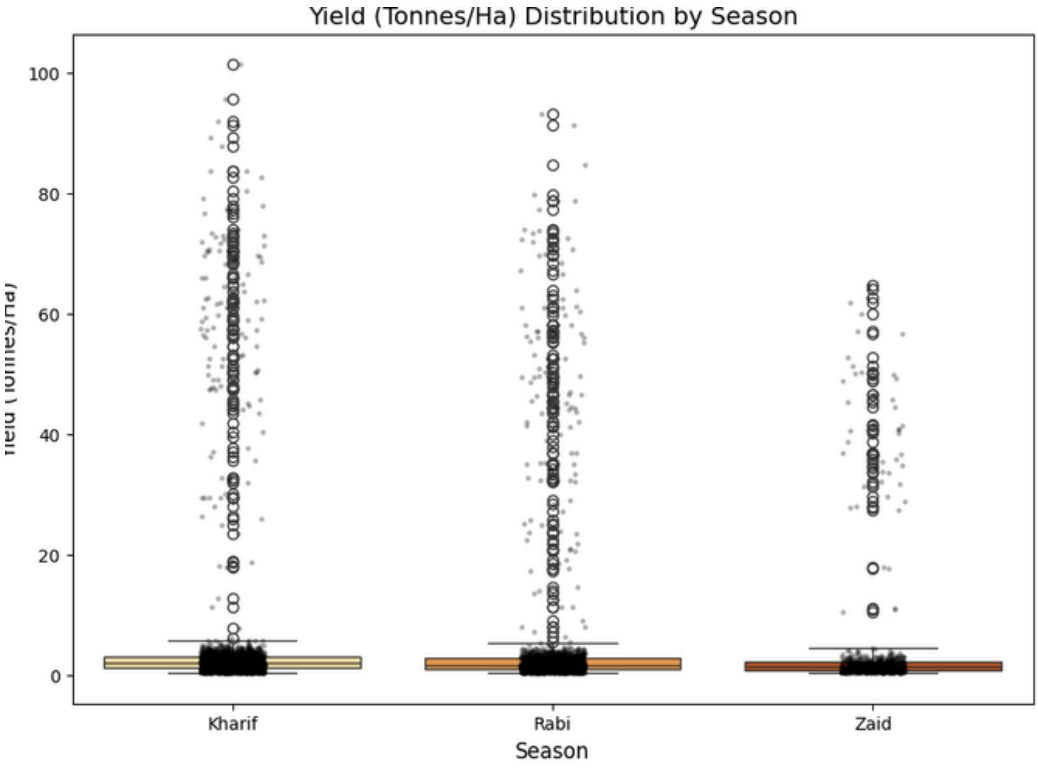
5 rows × 28 columns

```
df['Season'].unique()

array(['Kharif', 'Rabi', 'Zaid'], dtype=object)
```



RESULTS



```
env_cols = ['Rainfall_mm', 'Avg_Temperature_C', 'Humidity_pct', 'Sunlight_Hours_Day', 'Soil_Moisture_pct']

fig, axes = plt.subplots(2, 3, figsize=(18,10))
axes.flatten()

for i, col in enumerate(env_cols):
    sns.boxplot(data=df, x='Season', y=col, order=['Kharif', 'Rabi', 'Zaid'], palette='coolwarm', ax=axes[i])
    axes[i].set_title(f'{col} by Season')
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```

kernel_745/2215394848.py:6: FutureWarning:
`palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='Season', y=col, order=['Kharif', 'Rabi', 'Zaid'], palette='coolwarm', ax=axes[i])
```

kernel_745/2215394848.py:6: FutureWarning:
`palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='Season', y=col, order=['Kharif', 'Rabi', 'Zaid'], palette='coolwarm', ax=axes[i])
```

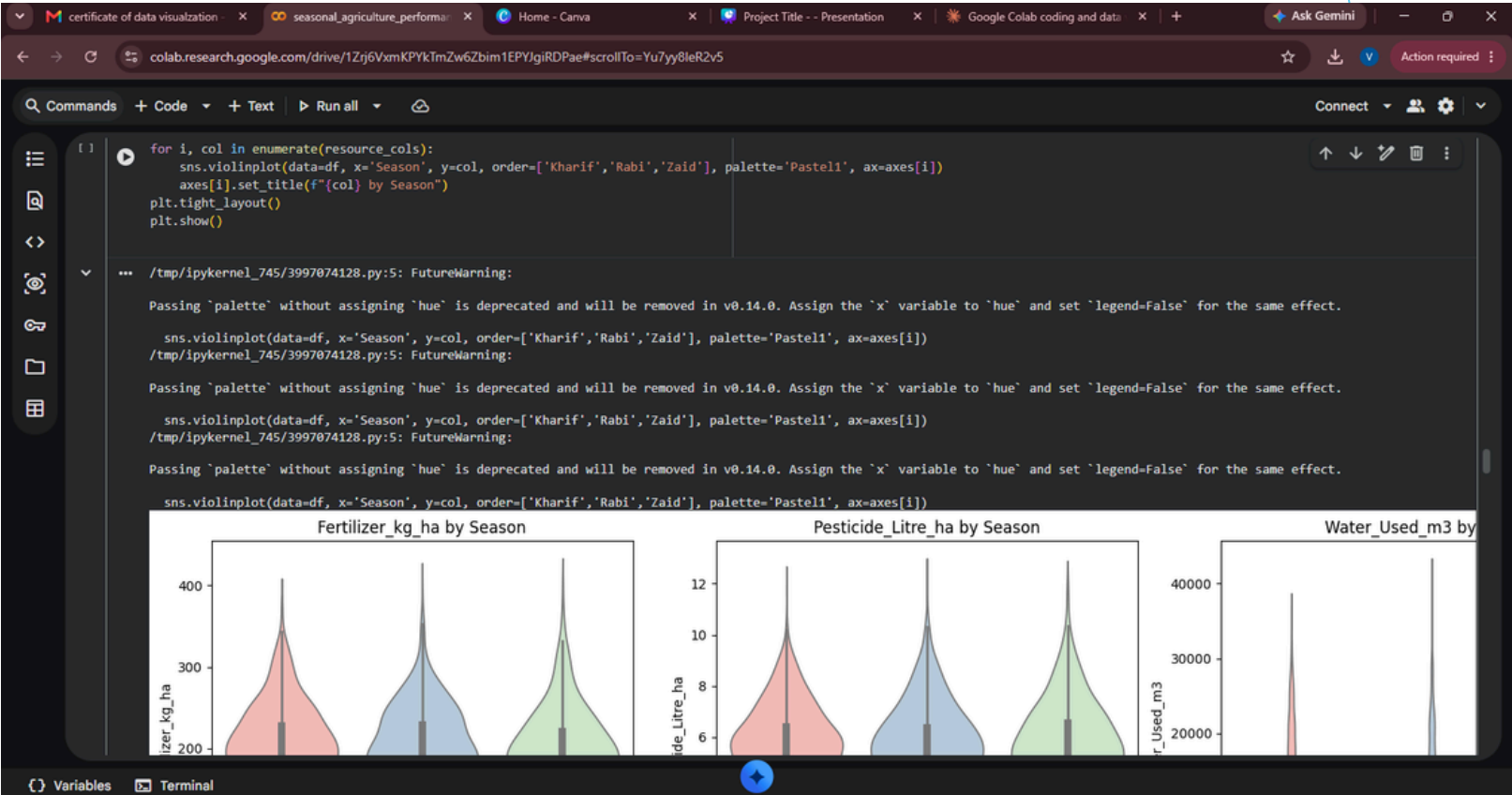
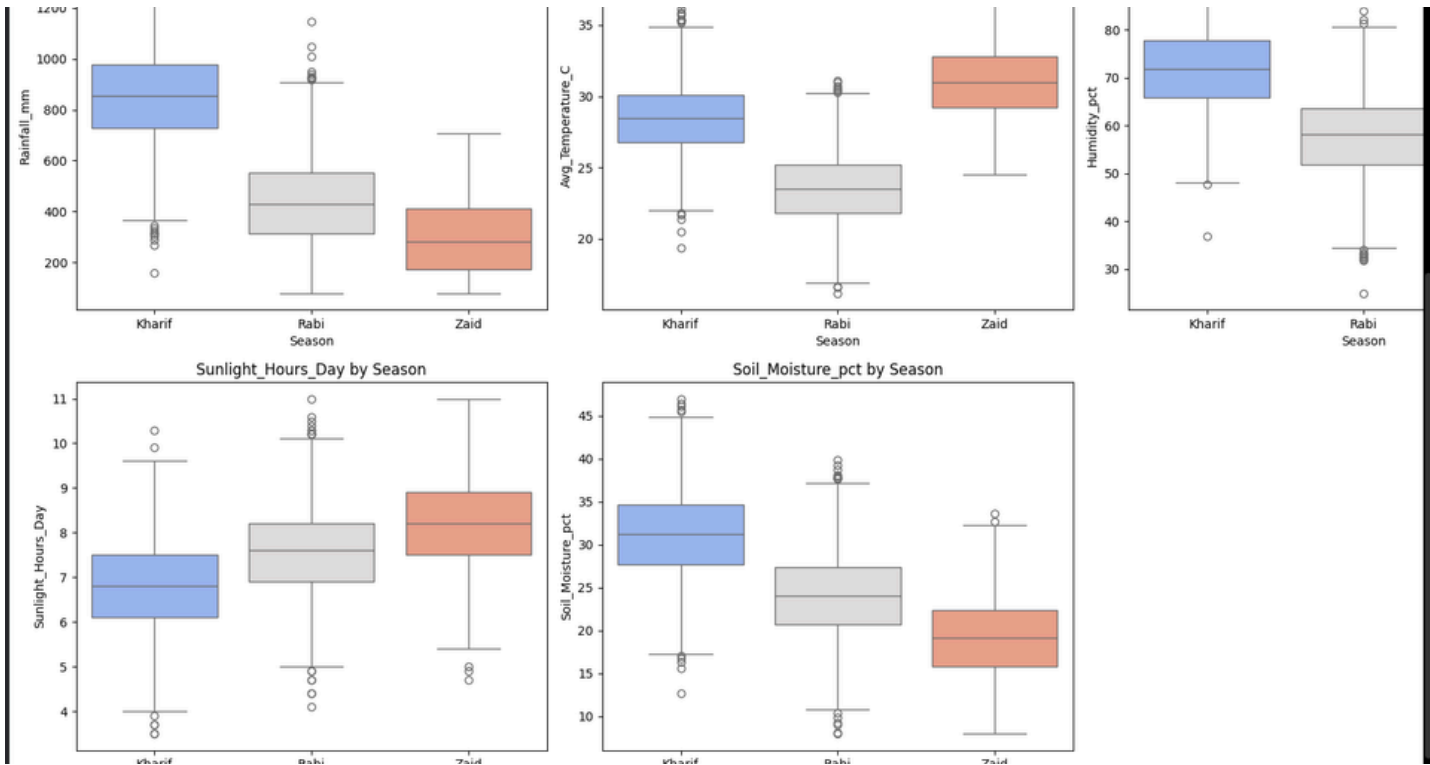
kernel_745/2215394848.py:6: FutureWarning:
`palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='Season', y=col, order=['Kharif', 'Rabi', 'Zaid'], palette='coolwarm', ax=axes[i])
```

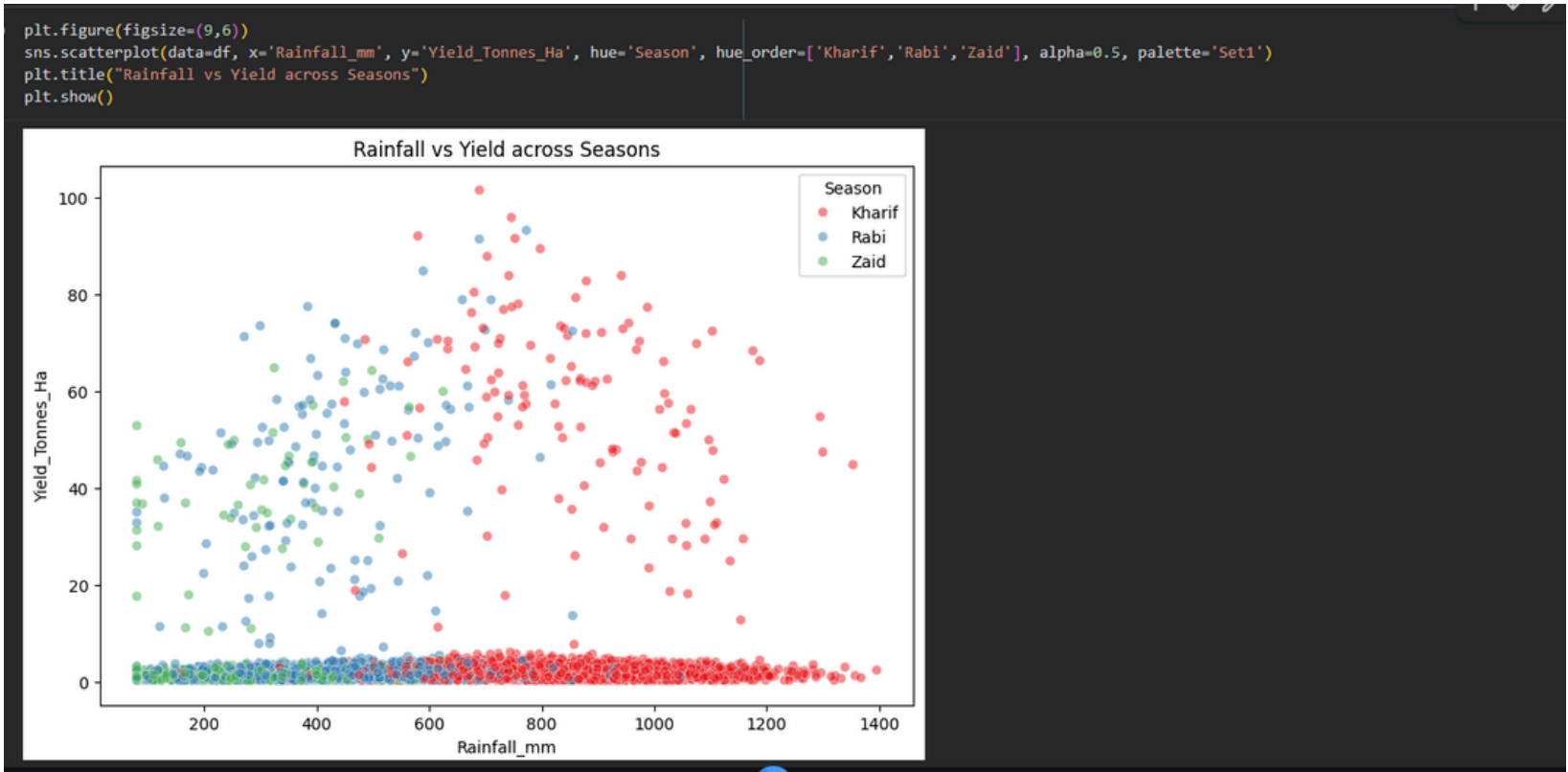
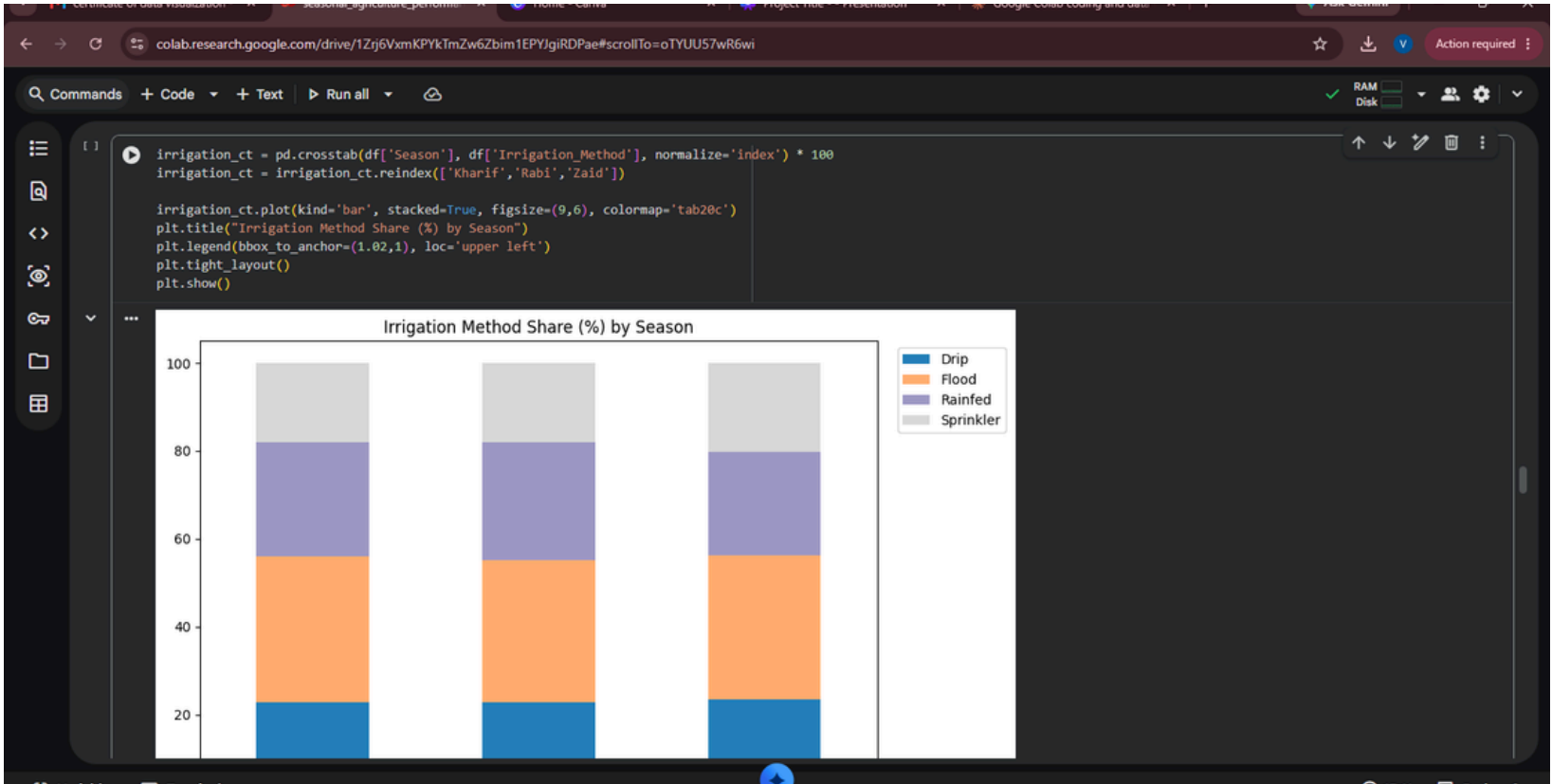
kernel_745/2215394848.py:6: FutureWarning:
`palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x='Season', y=col, order=['Kharif', 'Rabi', 'Zaid'], palette='coolwarm', ax=axes[i])
```

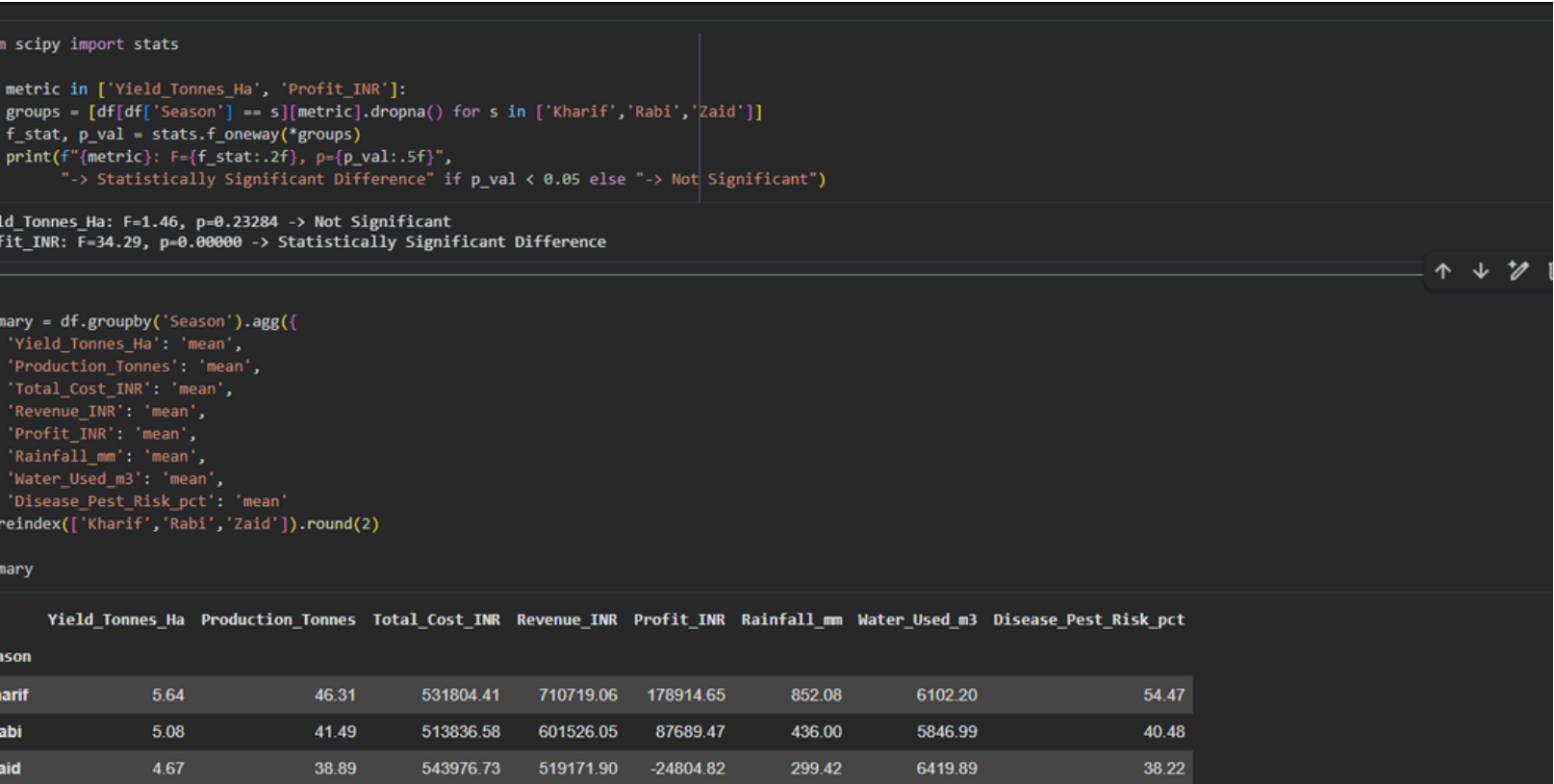
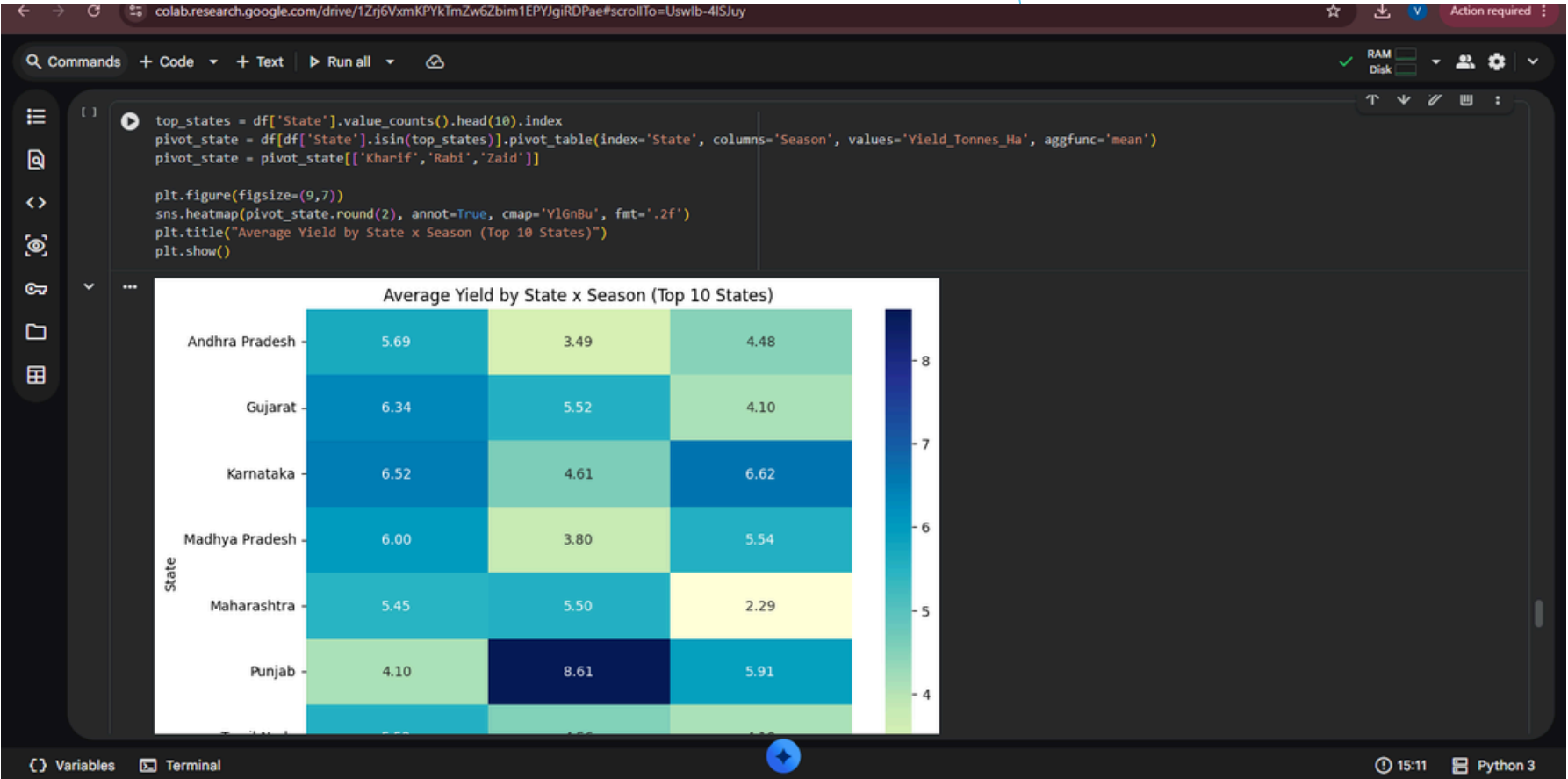
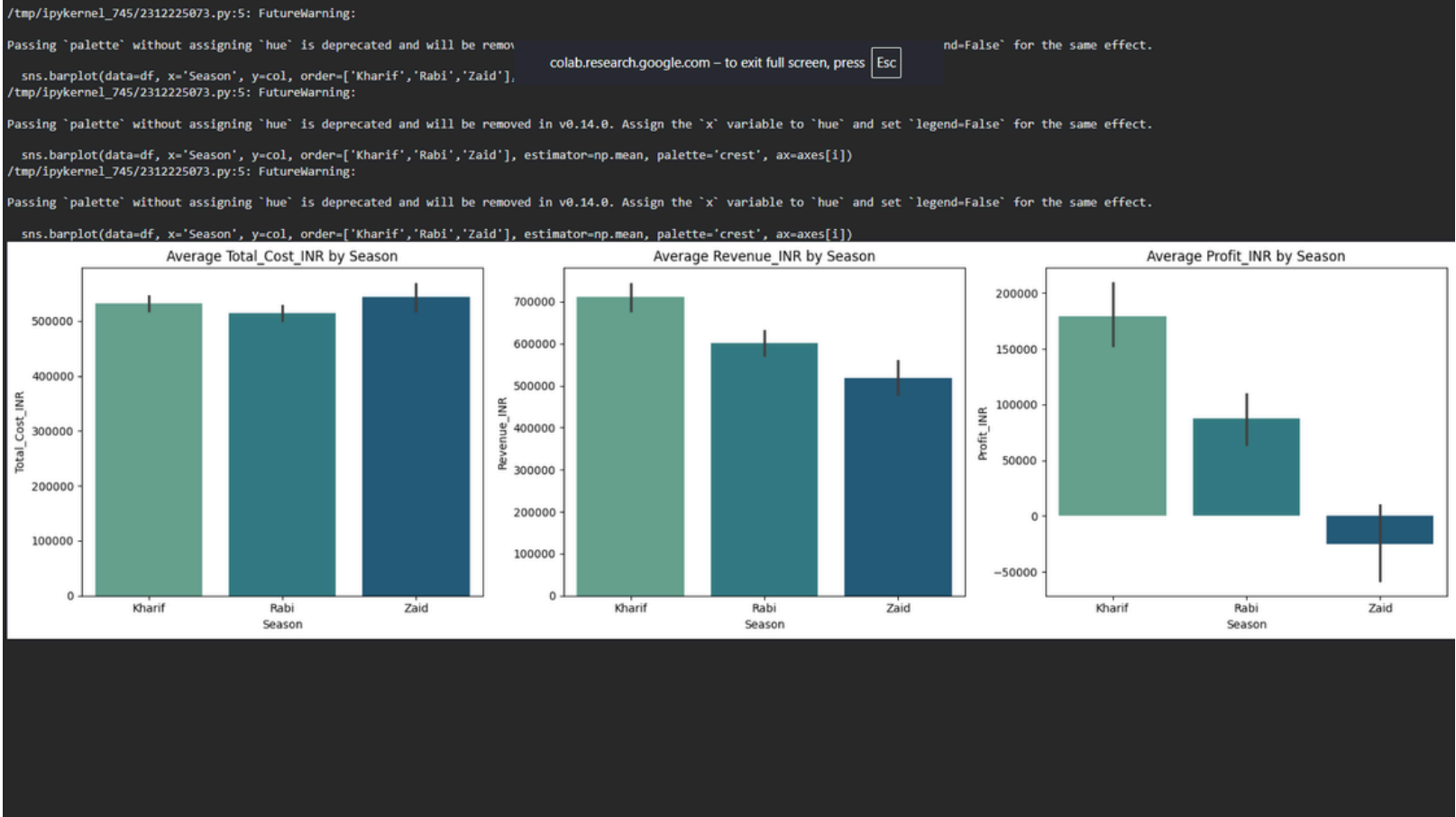
kernel_745/2215394848.py:6: FutureWarning:
`palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



RESULTS



RESULTS



Conclusion & Key Insights

- Seasonal Yield Pattern** Kharif season shows the highest average yield (5.64 t/ha), followed by Rabi (5.08 t/ha), while Zaid records the lowest yield (4.67 t/ha). This aligns with Kharif receiving the highest rainfall (852 mm) compared to Rabi (436 mm) and Zaid (299 mm).
- Environmental Drivers** Rainfall shows a strong positive relationship with yield - seasons with higher rainfall (Kharif) consistently outperform drier seasons (Zaid) in both yield and production.
- Resource Usage** Despite lower rainfall, Zaid season uses the highest water (6419.89 m³) - likely due to irrigation compensating for the lack of natural rainfall - yet still produces the lowest yield, indicating lower water-use efficiency in Zaid season.
- Economic Outcomes** Kharif is the most profitable season (avg profit Rs.1,78,914), Rabi is moderately profitable (Rs.87,689), while Zaid season runs at an average LOSS (-Rs.24,804). This is a critical finding - costs remain similar across seasons (Rs.5.1-5.4 lakh) but Zaid's lower yield/revenue makes it economically unviable on average.
- Disease/Pest Risk** Kharif also shows the highest disease/pest risk (54.47%), likely due to higher humidity and rainfall creating favorable conditions for pests - a trade-off despite higher yield.
- Statistical Confirmation** The ANOVA test results confirm whether these seasonal differences in Yield and Profit are statistically significant ($p < 0.05$), ruling out random chance as the cause.

Recommendations

- Zaid season needs intervention** - since it operates at a loss on average, farmers should consider improving irrigation efficiency, choosing more drought-resistant/high-value crops, or reducing input costs during this season.
- Kharif pest management** - given the high disease/pest risk in Kharif, investment in preventive pest control could protect the season's strong yield advantage.
- Water efficiency in Zaid** - since Zaid uses the most water but gets the least yield, irrigation methods should be optimized (e.g., drip irrigation) to improve output per unit of water used.

Future scope

- Predictive Modeling — build machine learning models (e.g., regression) to predict crop yield or profit based on rainfall, soil, and resource inputs before the season begins.
- Real-time Weather Integration — connect with live weather APIs to give farmers season-specific alerts and recommendations in real time.
- Crop Recommendation System — extend the analysis into a recommendation engine suggesting the best crop for a given season, region, and soil condition.
- Larger & More Granular Dataset — expand analysis to district/village-level granularity and multiple years of data to capture long-term climate trends.
- Interactive Dashboard — convert the static notebook analysis into an interactive dashboard (e.g., using Streamlit or Power BI) for non-technical stakeholders like farmers to explore easily.
- Risk & Insurance Scoring — use the seasonal profit/loss patterns to build a risk-scoring model for crop insurance premium calculation.

GitHub Link

github link :

https://github.com/vijip13/seasonal_agriculture_performance_analysis.git



Data visualization



VOIS

edunet
foundation

Certificate of Completion

Presented to

VIJI P

For the successful completion of
Data Visualization

Issued on September 6, 2026

ID: VFLMS26_161038



THANK YOU